DATA SCIENCE & ANALYTICS · SESSION 01

Data Science from the Ground Up

How raw data becomes decisions — and how to build that in Python

Asst.Prof.Dr. Chaiwat N. · 2026

2026 · Bachelor Program · 3 hrs · Workshop Format · Hands-on Coding

Session Agenda

01 **Course Orientation**

The course, the 13-session arc, how you're graded, the project spine

02 **The Data Science Workflow**

Six stages from a question to a decision, unpacked one at a time

03 **Your Environment**

Google Colab, your first notebook, verifying the setup

04 **Python Crash Course**

Variables, collections, loops, functions — coded live

05 **First Pandas Look**

The DataFrame, read_csv, and the four inspection commands

06 **Workshop Block**

Load a real CSV and print five facts about your data

01

PART 01

Course Orientation

What this course is, how the 13 sessions build on each other, how you'll be graded, and the project that ties everything together.

Data science is one skill: turning raw data into decisions someone can act on

What you're signing up for

What it is	Using data to answer real questions and drive decisions — not statistics for its own sake, not coding for its own sake. The question always comes first.
What you'll do	By Session 13 you can load any dataset, clean it, explore it, visualize it, build a simple model, and present a finding a non-technical person can act on.
How we work	Workshop format. You write code in every single session. Theory is kept short — the keyboard is where the learning actually happens.
Why it compounds	Each skill stacks on the last: Pandas feeds exploration, exploration feeds visualization, and all of it feeds your final project and a first taste of ML.

Source: DS Course, Session 1, 2026

How you're graded: four components, deliberately weighted toward applied work

Assessment · 15 / 25 / 20 / 40

In-class coding — 15%

Short graded notebooks during sessions. Rewards showing up and writing code, not memorizing slides. Miss the session, miss the marks.

Project presentation — 25%

Your team presents its end-to-end analysis. Clarity and story matter as much as the code — a finding nobody understands is worth nothing.

Project final report — 20%

Written deliverable: summary, data, method, findings, recommendation. The professional artifact you could hand a manager.

Final exam — 40%

Concepts plus applied. Part of it hands you a dataset and asks you to answer questions in code — exactly what you practice every week.

Source: DS Course, Session 1, 2026

The 13-session arc: foundations, then analytics, then real cases, then a first taste of ML

Where this is going

Phase 1 — Foundations (S01–S03)	Setup, Pandas manipulation, and data cleaning. The bedrock every later session stands on. Skip this and nothing downstream works.
Phase 2 — Explore & Tell (S04–S05)	Descriptive statistics, exploratory data analysis, and turning a pile of charts into a clear narrative.
Phase 3 — End-to-End Cases (S06–S09)	Two real business cases worked start to finish: sales performance analysis, then customer segmentation.
Phase 4 — Intro to ML (S10–S13)	Feature pipelines, regression, classification, and your final project pitch in front of the room.

Source: DS Course, Session 1, 2026

02

PART 02

The Data Science Workflow

Six stages that every data project moves through — from a vague question to a result someone can act on. Master the loop once and reuse it for every dataset.

Every data project follows the same six-stage loop — learn it once, reuse it forever

Question → Data → Clean → Analyze → Model → Communicate

Question · Data	Define what you're actually trying to learn, then find and load the data that can answer it. A sharp question is half the work; fuzzy questions sink projects.
Clean	Fix missing values, wrong types, and duplicates. In real work most of the time lives here — not in the modeling everyone imagines.
Analyze · Model	Summarize, visualize, and find patterns; then, optionally, fit a model to predict or to group. Description before prediction, always.
Communicate	Translate the result into a chart and a sentence a decision-maker understands. An analysis nobody acts on changed nothing.

Source: DS Course, Session 1, 2026

Stage 1 — Question: the sharper the question, the easier every later stage becomes

Define what you're actually asking

What you're doing	Turning a vague ask ('how's the business doing?') into a precise, answerable question ('which 10 products drove the most revenue last quarter?').
Sharp vs. vague	'Are sales good?' has no answer. 'Did revenue grow month-over-month in 2024, and which region led?' tells you exactly what data and chart you need.
Why it matters	The question decides the data, the cleaning, and the chart. Get it wrong and every downstream hour is wasted on the wrong thing.
The trap	Jumping to code before the question is clear. Five minutes defining the question saves five hours of aimless analysis.

Source: DS Course, Session 1, 2026

Stage 2 — Data: getting raw numbers off disk and into a DataFrame you can compute on

Find it, load it, confirm it

What you're doing	Reading CSVs, querying databases, or pulling from an API to get raw data into a pandas DataFrame — the table Python can actually work with.
Common sources	CSV and Excel files, SQL databases, and web APIs. This course starts with CSVs because they're everywhere and need no credentials.
First check	Always confirm the load: df.shape for size, df.head() for a peek. A silent wrong-file load is the most common beginner mistake.
Watch out	Separators (comma vs. semicolon), encodings, and headers in the wrong row. If the shape looks absurd, the load went wrong — not the data.

Source: DS Course, Session 1, 2026

Stage 3 — Clean: where most of the real work hides and beginners least expect it

Make the data trustworthy

What you're doing	Handling missing values, fixing wrong data types, removing duplicates, and standardizing categories so the numbers can be trusted.
Missing values	Decide per column: drop the row, fill with a sensible value, or leave it flagged. Each choice changes your result — choose on purpose.
Wrong types	A price stored as text won't sum; a date stored as text won't sort. Check df.info() and convert before you analyze, not after.
Reality check	Surveys say data scientists spend ~60–80% of their time here. Budget for it — clean data is most of the job, not a detour.

Source: DS Course, Session 1, 2026

Stage 4 — Analyze: summarize, visualize, and let the patterns show themselves

Describe before you predict

What you're doing	Computing summary statistics, grouping and aggregating, and plotting distributions and relationships to understand the data's shape.
Core moves	groupby to compare segments, describe() for the five-number summary, and a histogram or scatter to see what a table hides.
Let the chart talk	A single well-chosen plot often reveals an outlier, a trend, or a data error faster than any table of numbers.
Watch out	Averages lie when the distribution is skewed. Always look at the spread, not just the mean — one whale customer can distort everything.

Source: DS Course, Session 1, 2026

Stage 5 — Model: move from describing what happened to predicting or grouping

Optional, but where ML begins

What you're doing	Fitting a simple model — regression to predict a number, classification to predict a label, or clustering to group similar records.
When you need it	Only after description. A model on dirty or misunderstood data produces confident nonsense — the most dangerous output of all.
This course	You'll meet pipelines, regression, and classification in Phase 4. Today you only need to know modeling is one stage, not the whole job.
Golden rule	A model is a tool to answer the question from Stage 1 — never the goal in itself. Simple and understood beats complex and opaque.

Source: DS Course, Session 1, 2026

Stage 6 — Communicate: an analysis nobody understands changes exactly nothing

Turn the result into a decision

What you're delivering	A chart with labeled axes and a clear title, plus one or two sentences: what you found, why it matters, what to do about it.
The three questions	What did you find? (description) Why does it matter? (implication) What should we do? (recommendation). Every output needs all three.
Title = insight	'Monthly Revenue' is a label. 'Revenue fell 18% after the March price change' is a title that makes the point for you.
Why it's graded	Your project presentation is 25% and the report 20% — because in the real world, communication is where analysis becomes impact.

Source: DS Course, Session 1, 2026

03

PART 03

Your Environment

Google Colab, your first notebook, and the habit of verifying your setup before you write a single line of analysis.

Google Colab: a full data-science environment in the browser, nothing to install

Where you'll write every notebook

What it is	A free, browser-based Jupyter notebook from Google. Python, pandas, and the whole toolkit come pre-installed — zero setup, any laptop.
Why we use it	No 'it works on my machine' problems. Everyone in the room runs the identical environment, so class never stalls on installation.
How to open	colab.research.google.com → New notebook → sign in with your university Google account. That's the entire setup.
Cells & run	A notebook is a stack of cells. Write code in a cell, press Shift+Enter to run it, and the output appears right below. Code and results live together.

Source: DS Course, Session 1, 2026

Verify your setup before analysis: a working environment is the first deliverable

Run this first cell

Check it runs

If this cell prints versions without an error, your environment is ready and every later session will work.

Read the output

pandas ≥ 1.5 and Python 3.9+ is all you need. A NameError or ImportError means something didn't load — fix it now, not mid-exercise.

Pair up if stuck

If your environment is broken, pair with a neighbour and keep moving. Never hold up the whole class on one install.

PYTHON · FIRST CELL

```
import sys
import pandas as pd
import numpy as np

print('Python :', sys.version.split()[0])
print('pandas :', pd.__version__)
print('numpy  :', np.__version__)

# Expected output:
# Python : 3.10.12
# pandas : 2.0.3
# numpy  : 1.25.2
```

If this runs clean, you are ready for the workshop.

04

Python Crash Course

The four building blocks you'll use in every notebook for the rest of the course: variables and types, collections, loops and conditionals, and functions.

PART 04

Variables & types: every value Python holds has a type, and the type sets the rules

The four you'll use daily

Variable

A name pointing at a value. = is assignment, not equality. Names are cheap — use clear ones like revenue, not r.

The core types

int and float are numbers, str is text, bool is True/False. type(x) tells you which — check it when maths behaves oddly.

Why it matters

'100' + '50' makes '10050', but 100 + 50 makes 150. Type confusion is the #1 source of silent beginner bugs.

PYTHON · VARIABLES

```
price = 100         # int
growth = 0.18       # float
product = 'Laptop'  # str
in_stock = True     # bool

print(type(price))     # <class 'int'>
print(type(growth))    # <class 'float'>

# numbers add, strings concatenate
print(price + 50)      # 150
print('A' + 'B')       # 'AB'
```

type(x) reveals how a value will behave.

Collections: lists hold an ordered sequence, dicts map a key to a value

How you store many values

List

An ordered, changeable sequence in []. Access by position, starting at 0. Use it for 'a bunch of things in order'.

Dict

A set of key → value pairs in {}. Access by key, not position. Use it for 'look this up by name'.

Why both

A DataFrame is essentially a dict of columns, each column a list-like sequence. These two ideas underpin all of pandas.

PYTHON · LIST & DICT

```
# list: ordered, index from 0
prices = [100, 250, 80]
print(prices[0])      # 100
prices.append(60)     # add one

# dict: key -> value
product = {
    'name': 'Laptop',
    'price': 100,
}
print(product['price'])   # 100
```

Lists index by position; dicts look up by key.

Loops & conditionals: do something for every item, and branch on a condition

Repeat and decide

for loop

Runs a block once per item in a sequence. This is how you process every row, file, or product without copy-pasting.

if / else

Chooses a path based on a True/False test. Combine with a loop to act differently on each item.

Watch out

Indentation is not cosmetic in Python — it defines what's inside the loop or the if. Misaligned code is a different program.

PYTHON · LOOP & IF

```
prices = [100, 250, 80]

for p in prices:
    if p >= 100:
        print(p, 'expensive')
    else:
        print(p, 'cheap')

# 100 expensive
# 250 expensive
# 80 cheap
```

Indentation defines the block — keep it consistent.

Functions: name a block of logic once, then reuse it everywhere without repeating

Package logic for reuse

Define with def

A function takes inputs (parameters), does work, and returns a result. Write the logic once; call it as many times as you like.

Why it matters

Functions kill copy-paste. Fix a bug in one place and every call is fixed — the foundation of clean, trustworthy notebooks.

You already use them

print(), len(), pd.read_csv() are all functions. You're just learning to write your own.

PYTHON · FUNCTIONS

```
def discount(price, pct):
    """Return price after a % discount."""
    return price * (1 - pct)

print(discount(100, 0.20))   # 80.0
print(discount(250, 0.10))   # 225.0

# write once, call many times
# fix once, fixed everywhere
```

def names logic once so you never repeat it.

05

PART 05

First Pandas Look

The DataFrame mental model, loading a file with read_csv, and the four commands you run on every dataset you will ever open.

The DataFrame: a spreadsheet that Python can compute on, at any scale

The one object you'll use all course

What it is	A 2-D table of rows and columns — like a sheet in Excel, but driven by code so it scales from 50 rows to 50 million.
Rows & columns	Each row is a record (one sale, one customer); each column is a field (price, date, region) and has a single data type.
The index	Every row has a label — the index. Default is 0,1,2,…, but it can be a date or an ID, which makes lookups and joins fast.
dtypes matter	Each column knows its type: numbers compute, text concatenates, dates sort and resample. Wrong dtype, wrong answer — always check.

Source: DS Course, Session 1, 2026

read_csv: loading a file is one line — but always confirm what you actually got

From file to DataFrame

One line to load

pd.read_csv() reads a file into a DataFrame. Point it at a path or URL and pandas infers columns and types.

Then verify

df.shape tells you rows × columns. If the numbers look wrong, suspect the separator or a header row — not the data.

Common args

sep for the delimiter, parse_dates to read dates as real dates. Small flags that save hours of cleaning later.

PYTHON · READ_CSV

```
import pandas as pd

df = pd.read_csv('sales.csv')

df.shape
# (1500, 6)  -> 1500 rows, 6 columns

# if dates are text, parse them:
df = pd.read_csv(
    'sales.csv',
    parse_dates=['date'],
)
```

Always run df.shape right after loading.

The four commands you run on every single dataset, in this order, before anything else

Your first-look checklist

.head()	Shows the first five rows. Your sanity check that the file loaded correctly and the columns are what you expected.
.info()	Lists every column, its dtype, and how many non-null values it has. This is where you spot missing data and wrong types instantly.
.describe()	Summary statistics — count, mean, min, max, quartiles — for the numeric columns. Reveals outliers and impossible values fast.
.shape · .columns	Size of the table and the list of column names. The two-second orientation you do before writing any analysis.

Source: DS Course, Session 1, 2026

06

Workshop Block

60 minutes hands-on: open Colab, load a real sales CSV, inspect it, and print five facts about your data — your first graded in-class notebook.

PART 06

By the end of this block, every student has loaded real data and printed five facts about it

4 Steps

Open Colab	colab.research.google.com → New notebook → sign in with your university account. Nothing to install. Stuck? Pair with a neighbour.
Load the data	Upload the provided sales.csv, then df = pd.read_csv('sales.csv'). Confirm it loaded with df.shape before doing anything else.
Inspect	Run df.head(), df.info(), and df.describe(). Read the dtypes carefully — are the numbers actually stored as numbers?
Print five facts	Rows, columns, the date range, the number of unique products, and total revenue. This is your graded deliverable.

Source: Workshop, Session 1, 2026

Your in-class deliverable: five facts about the data in one clean, readable cell

What 'done' looks like

The task

Print exactly five facts. Clear labels, correct numbers, one cell that runs top to bottom without error.

Graded on

Did it run? Are the numbers right? Is it readable? Clean, labeled output beats clever one-liners every time.

If you finish early

Add a sixth fact of your own — and try plotting one column with df['revenue'].hist(). Curiosity is the whole skill.

PYTHON · DELIVERABLE

```
import pandas as pd
df = pd.read_csv('sales.csv',
parse_dates=['date'])

print('Rows          :', df.shape[0])
print('Columns       :', df.shape[1])
print('Date range    :', df['date'].min(),
      '->', df['date'].max())
print('Products      :', df['product'].nunique())
print('Total revenue:', df['revenue'].sum())
```

Five facts, one cell, runs clean. That's the bar.

KEY TAKEAWAY

The workflow is the skill. The tools just execute it.